

# Tutorial Sheet 5b

## Inter-Process Communication

AUTHOR

Operating Systems Teaching and Research Unit

PUBLISHED

28.06.2024

## Introduction Questions

### Question 1

Pipes and Shared Memory are two concepts for exchanging information between processes. Compare the advantages and disadvantages of these two concepts.

### Question 2

In the lecture, it was mentioned that named pipes are very similar to files. What are the main differences between files and named pipes?

### Question 3

#### Scenario: Web Server Configuration

Web servers typically read a configuration file and apply rules based on it while running. To avoid restarting the server every time the configuration changes, we aim to implement a feature that allows us to update the configuration and trigger a reload without restarting.

Discuss the advantages and disadvantages of using different methods of inter-process communication to achieve this:

- Signals
- Shared Memory
- Pipes
- Sockets

### Question 4

#### Scenario: Autonomous Vehicle Control System

In an autonomous vehicle control system, various subsystems must exchange real-time data such as sensor readings, decision outputs, and control commands. The system requires low latency for rapid response to dynamic road conditions, high bandwidth to handle continuous sensor inputs, and effective synchronization to ensure timely decision-making and execution of control commands.

Discuss the advantages and disadvantages of using each method of IPC to achieve efficient data exchange and synchronization between subsystems in an autonomous vehicle control system.

- Signals
- Shared Memory

- Pipes
- Sockets

## Question 1

- **Pipes:** Defined access via `read()` and `write()`. With “proper” usage (unidirectional, each end closing one side of the pipe), no special synchronization is required. Disadvantages include overhead in communication: `read()` and `write()` are both syscalls, causing context switches, and there's memory content copying.
- **Shared Memory:** Significantly higher performance due to no memory copying during data exchange, as both processes use the same physical memory area. Disadvantage is the need for synchronization by the programmer to avoid race conditions or inconsistencies.

## Question 2

Named pipes do not store data in a file on a file system. Once data is read, it cannot be read again and essentially “disappears”. Named pipes do not support searching like files; data access is serial. Hence, the use of the term FIFO (first in, first out) for named pipes, as seen in commands like `mkfifo`.

## Question 3

### 1. Signals

Signals are a very good choice in this context. They are very easy to implement and to use, we don't even need to write new code for this, we can just use the `kill` command to send a signal to our main program. The only code to write is the signal handler.

### 1. Shared Memory

Shared Memory is overkill in this context. We need to write a full new program with shared memory, we need the main program to wait on a semaphore to check if the configuration reload is needed. In addition, we don't need to transmit a lot of data, so the use of shared memory is not justified in this context.

### 1. Pipe

We need to create a named pipe to communicate with the main program. The advantages over shared memory is that we don't need synchronization anymore, and it's easier to write. But this solution is still expensive and there is the overhead of checking for the content of the pipe regularly.

### 1. Socket

Socket are only useful if we want communication over the network. On the same machine, sockets are unnecessarily complex compared to pipes or shared memory.

## Question 4

### 1. Signals

Signals are lightweight and efficient for notifications or triggers. They are easy to implement and can be used for notifications between subsystems about events. However, they are limited and don't allow complex data exchange. This can be problematic in your system that requires detailed data sharing. Also,

the latency of signals can be high, as there are no guarantee when the signal will be sent to the process by the operating system.

#### 1. Shared Memory

Shared memory lets processes access data directly, minimizing delays and supporting high data throughput critical for real-time exchanges in autonomous vehicles. It needs careful synchronization to prevent data mix-ups or system freezes. Although managing shared memory can be tricky, it offers big performance gains for handling large data loads.

#### 1. Pipes

Pipes are simple for one-way communication between processes. They work well for passing data step by step but add extra work for handling big chunks of data. While pipes are easy to set up, they only allow one-way talk and don't do so well with the big data needs of self-driving cars.

#### 1. Sockets

Sockets help with local and networked talking, giving you flexibility in how your setup works. They're reliable and can scale up as you grow. However, they do take longer to send messages because they have to go through the network and take time to get set up again.

## Tasks

### Task 1: Named Pipes

In the lecture, you learned about named pipes as a mechanism for IPC. Write two programs, `sender.c` and `receiver.c`, that communicate with each other through a named pipe. The `sender.c` program should continuously accept user input and then send it to `receiver.c`, which should continuously wait for transmissions and then print them out. Both programs should terminate when `exit` is transmitted.

## Solution

The implementation is relatively simple, with error handling being the only somewhat annoying part. It's best to implement it first without any error handling. It should compile without warnings using `-std=c17 -Wall -Wextra -pedantic`. Afterward, go through the code together and consider where errors might occur. It is sufficient to implement a few error fixes as examples.

- **mkfifo**: Path already exists or the folder does not exist.
- **open**: No permissions for the file or the file does not exist.
- **write**: The file descriptor is not writable (e.g., no write permissions) or the buffer cannot be accessed because it is outside the address space.
- **read**: The file descriptor is not readable, or there is an I/O error, for example, because the USB stick was removed.
- **close**: The file descriptor is invalid, or there is an I/O error.
- **remove**: File does not exist.

A complete list of possible errors can, of course, be found in the corresponding man pages. Feel free to demonstrate this with an example.

receiver.c:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <fcntl.h>
5  #include <sys/stat.h>
6  #include <unistd.h>
7
8  #define BUFFER_SIZE 256
9
10 int main ()
11 {
12     char mypipe [] = "/tmp/mypipe";
13     int fd_recv = open(mypipe , O_RDONLY);
14     if (fd_recv == -1) {
15         perror("open");
16         exit (1);
17     }
18     char buffer_recv[BUFFER_SIZE ];
19     do {
20         if (read(fd_recv , buffer_recv , sizeof(buffer_recv)) == -1) {
21             perror("read");
22             exit (1);
23         }
24         printf("> %s", buffer_recv);
25     } while (strcmp(buffer_recv , "exit\n"));
26     if (close(fd_recv) == -1){
27         perror("close");
28         exit (1);
29     }
30     return 0;
31 }
```

sender.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <fcntl.h>
5  #include <sys/stat.h>
6  #include <unistd.h>
7
8  #define BUFFER_SIZE 256
9
10 int main ()
11 {
12     char
13     mypipe [] = "/tmp/mypipe";
14     if (mkfifo(mypipe , S_IRUSR | S_IWUSR) == -1) {
15         perror("mkfifo");
16         exit (1);
17     }
18     int fd_send = open(mypipe , O_WRONLY);
19     if (fd_send == -1) {
20         perror("open");
21         exit (1);
22     }
23     char buffer_send[BUFFER_SIZE];
24     do {
25         fgets(buffer_send , BUFFER_SIZE , stdin);
26         if (write(fd_send , buffer_send , strlen(buffer_send)+1) == -1){
27             perror("write");
28             exit (1);
29         }
30     } while (strcmp(buffer_send , "exit\n"));
31     if (remove(mypipe) == -1) {
32         perror("remove");
33         exit (1);
34     }
35     if (close(fd_send) == -1) {
36         perror("close");
37         exit (1);
38     }
39     return 0;
40 }

```

## Task 2: Shared and Synchronized Blocked Stack

Write in C a shared stack supporting the pop and push operations.

Pop is used to remove the element on the top of the stack. If no element is available, wait until one becomes available.

Push is used to add an element on the top of the stack. If the stack is full, **push** should wait until an element can be inserted.

Multiple processes can push and pop at the same time. Your implementation should be safe meaning that multiple processes/threads can execute the code at the same time without race conditions.

```
1 struct Stack {
2     int top; // the index of the element above the top of the stack
3     int size;
4     int *stack;
5     // complete here with synchronization mechanisms
6 };
7
8 struct Stack *init_stack(int size) {
9     // allocate and initialize the stack
10 }
11
12 int pop(struct Stack *stack) {
13     // pop the top element of the stack and return it
14 }
15
16 void push(struct Stack *stack, int value) {
17     // push a new element on the top of the stack
18 }
```

## Solution

The shared memory can also be used with the `MAP_SHARED` and `MAP_ANONYMOUS` flags of `mmap`, instead of using `shm_open`: the API is easier but it won't work across different processes, only for parents and children.

`MAP_SHARED` defines a memory area that can be shared between processes after fork.

`MAP_ANONYMOUS` is used for memory allocations (like `malloc`).

If you have questions with the difference between `malloc` and `mmap`:

- `malloc` is using `mmap` under the hood
- `malloc` is optimized so only one big `mmap` is made, and other `malloc` manage the memory allocations in user space

If you have questions with `MAP_ANONYMOUS`

- `mmap` means “memory map”, literally creating a new virtual address mapping of memory between either the main memory (page frames) or a file
- `mmap` can be used to map a file to the virtual memory of a process, by providing a file descriptor and without the flag `MAP_ANONYMOUS`

The solution works like this:

- a mutex to protect the `idx` variable and the content of the stack
- a semaphore for the number of free elements in the stack
- a semaphore for the number of used elements in the stack

The use of semaphores is allowing the stack to be blocking: if no elements can be inserted/removed then we wait because it's how the semaphore behaves.



```

1  struct Stack {
2      int idx;
3      int size;
4      sem_t sem_free;
5      sem_t sem_used;
6      sem_t mutex;
7      int *stack;
8  };
9
10 struct Stack *init_stack(int size) {
11     struct Stack *stack = mmap(NULL, sizeof(struct Stack), PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);
12     int *stack_t = mmap(NULL, sizeof(int) * size, PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);
13
14     if (!stack || !stack_t) {
15         perror("mmap");
16         exit(1);
17     }
18
19     stack->idx = 0;
20     stack->size = size;
21     sem_init(&stack->sem_free, 1, size);
22     sem_init(&stack->sem_used, 1, 0);
23     sem_init(&stack->mutex, 1, 1);
24
25     stack->stack = stack_t;
26     return stack;
27 }
28
29 int shared_pop(struct Stack *stack) {
30     sem_wait(&stack->sem_used);
31
32     sem_wait(&stack->mutex);
33     int ret = stack->stack[--stack->idx];
34     sem_post(&stack->mutex);
35
36     sem_post(&stack->sem_free);
37     return ret;
38 }
39
40 void shared_push(struct Stack *stack, int value) {
41     sem_wait(&stack->sem_free);
42
43     sem_wait(&stack->mutex);
44     stack->stack[stack->idx++] = value;
45     sem_post(&stack->mutex);
46
47     sem_post(&stack->sem_used);
48 }
49
50 int main() {
51     srand(time(NULL));
52
53     struct Stack *stk = init_stack(200);
54
55     for (int i = 0 ; i < 10; i++) {
56         if (fork()==0) {
57             while (1) {
58                 shared_push(stk, rand() % 20);
59             }
60         }

```

```
61
62     }
63
64     for (int i = 0 ; i < 10; i ++ ) {
65         if (fork()==0) {
66             while (1) {
67                 shared_pop(stk);
68             }
69         }
70     }
71
72     wait(NULL);
73 }
```